

WebAuthn PRF-Based Vault Key Wrapping and Zero-Knowledge PWA Storage Architecture

Ian Pinto¹

Independent Researcher, Mumbai, India
ianpinto1980@gmail.com

Abstract. This paper presents a client-side security architecture that combines the Web Authentication (WebAuthn) pseudo-random function (PRF) extension with symmetric cryptography and carefully engineered browser storage patterns to achieve strong zero-knowledge properties for web-based credential vaults. The design is grounded in the TrustVault-PWA project and informs the reusable `webauthn-prf-zktv` library.

Keywords: WebAuthn, PRF, passkeys, zero-knowledge, IndexedDB, PWA, cryptography

1 Introduction

The Web Authentication (WebAuthn) standard and FIDO2 ecosystem provide strong public-key based authentication and passkeys that are increasingly used to replace passwords on the web. Recent extensions such as the pseudo-random function (PRF) allow a relying party to request a high-entropy symmetric secret derived from a passkey credential during an authentication ceremony, enabling end-to-end encryption of user data.[?,?]

Password managers and credential vaults seek to offer zero-knowledge security: server-side or client-side storage alone should never be sufficient to reveal user secrets. Achieving this property in a web progressive web application (PWA) requires careful coordination of cryptography, browser APIs, and local storage. This paper formalizes a PRF-based vault key wrapping scheme and an IndexedDB storage architecture designed toward this goal.

1.1 Contributions

The main contributions are:

- A formal description of a WebAuthn PRF-based vault key wrapping scheme suitable for PWAs.
- An IndexedDB schema and migration strategy that reinforce zero-knowledge properties.
- A reusable design for the `webauthn-prf-zktv` library that packages these patterns.
- Discussion of post-quantum considerations and migration paths.

2 Background and Related Work

2.1 WebAuthn, Passkeys, and PRF

Summarize WebAuthn, FIDO2, passkeys, and the PRF extension. Reference the W3C WebAuthn specification and PRF extension documentation, as well as SimpleWebAuthn and Yubico developer guides.[?, ?, ?]

2.2 PRF-Based Vault Encryption in Existing Systems

Describe how Bitwarden and other systems use WebAuthn PRF to derive keys for vault encryption, highlighting similarities and differences with the approach in this paper.[?] Mention generic PRF helper libraries and file-encryption tools that integrate PRF without prescribing a full zero-knowledge vault architecture.[?]

2.3 IndexedDB and Client-Side Storage

Review IndexedDB as a browser storage mechanism and common patterns for encrypting secrets at rest. Explain how TrustVault-PWA uses Dexie-backed IndexedDB, encrypts both secrets and metadata, and documents residual risks such as breach-detection prefixes.[?]

3 Threat Model

Define the attacker capabilities:

- Stored-data compromise (IndexedDB dump, server backup).
- Browser compromise (XSS, supply-chain attacks).
- Authenticator or passkey compromise.
- Harvest-now, decrypt-later with future quantum capabilities.

Explain the security goals (zero-knowledge storage, bounded exposure in browser compromise, PQ-readiness).

4 PRF-Based Vault Key Wrapping

4.1 High-Level Design

Enrollment (`enrollVault`) and unlock (`unlockVault`) use the 32-byte WebAuthn PRF output as HKDF input keying material to derive a non-extractable AES-256-GCM *wrap key* K_w , which encrypts the 256-bit vault key K_v . The only persisted artifact is a `WrappedSecretRecord`:

$$R = (\text{scheme}, \text{ciphertext}, \text{nonce}, \text{salt}, [\text{kdfParams}])$$

where $\text{scheme} \in \{\text{prf-v1}, \text{pw-v1}\}$, the nonce is a fresh 96-bit random AES-GCM IV, and the salt is the PRF salt (`prf-v1`) or the scrypt salt (`pw-v1`). `kdfParams`

is present iff `scheme = pw-v1`; the parser (`parseRecord`) rejects `prf-v1` records carrying it. PRF outputs and wrap keys are never stored: wrap keys are created with `extractable: false` in Web Crypto, and transient PRF-output buffers are zeroized (`.fill(0)`) in `finally` blocks.

HKDF uses SHA-256 with an *empty* salt (valid per RFC 5869, treated as zeros); per-credential domain separation comes from the unique PRF salt, which yields unique input keying material. The HKDF `info` label is the frozen constant `"webauthn-prf-zktv vault key wrap v1"`; a legacy label `"TrustVault Vault Key Wrapping v1"` exists only in the one-shot TrustVault migration adapter (`fromTrustVaultRecord`), which unwraps under the legacy label and re-wraps under `v1` using the same PRF output, requiring no extra ceremony.

A password fallback scheme `pw-v1` covers clients where PRF is not viable (notably Android WebView, where PRF does not reach the Credential Manager): the wrap key is derived with memory-hard scrypt ($N=131072$, $r=8$, $p=1$ as a security floor) instead of HKDF-from-PRF; all other steps are identical.

4.2 Pseudo-Code

Algorithm 1 reflects `wrapSecret` in `src/core/wrap.ts` composed with the PRF ceremony (`enrollVault` in `src/webauthn/vault.ts`):

Algorithm 1 Vault key wrapping (`enrollVault` $\rightarrow$ `wrapSecret`, scheme `prf-v1`)

Require: WebAuthn credential C with PRF enabled, PRF salt s (32 random bytes), vault key K_v (32 bytes)

Ensure: Stored record R that does not reveal K_v without C

- 1: $prf \leftarrow \text{EvaluatePRF}(C, s) \triangleright 32$ bytes; `PrfResultMissingError` if absent or wrong length
 - 2: $K_w \leftarrow \text{HKDF-SHA256}(\text{ikm}=prf, \text{salt}=\varepsilon, \text{info}=\text{"webauthn-prf-zktv vault key wrap v1"})$
 - 3: import K_w as non-extractable AES-256-GCM `CryptoKey`
 - 4: $\text{nonce} \leftarrow \text{Random}(12)$
 - 5: $\text{ciphertext} \leftarrow \text{AES-256-GCM-Encrypt}(K_w, \text{nonce}, K_v)$ $\triangleright$ auth tag included
 - 6: zeroize prf $\triangleright$ `finally` block; K_w is non-extractable, never in JS memory
 - 7: $R \leftarrow (\text{scheme}=\text{prf-v1}, \text{ciphertext}, \text{nonce}, \text{salt}=s)$
 - 8: **return** R
-

Discrepancy: An earlier draft stated that wrapping zeroizes “any raw representation of K_v ”. In the implementation, `wrapSecret` deliberately does *not* zeroize the caller’s secret buffer — the caller owns K_v (it may still be needed, e.g. to also wrap under `pw-v1`). Zeroization inside the library is scoped to PRF outputs and to the transient plaintext produced during unwrap.

Algorithm 2 reflects `unlockVault` $\rightarrow$ `unwrapSecret`; note the replay verification step absent from generic treatments:

Algorithm 2 Vault key unwrapping (`unlockVault` $\rightarrow$ `unwrapSecret`)

Require: Credential id, stored record $R = (\text{prf-v1}, \text{ciphertext}, \text{nonce}, s)$, last stored signature counter c

Ensure: Non-extractable session `CryptoKey` K_v , new counter c'

- 1: $(\text{prf}, c') \leftarrow \text{EvaluatePRF}(C, s)$ $\triangleright$ assertion ceremony with fresh 32-byte challenge
- 2: verify `clientData`: `type=webauthn.get`, challenge and origin match; verify $c' > c$ (unless $c'=0$); else throw `ReplayError`
- 3: $K_w \leftarrow \text{HKDF-SHA256}(\text{ikm}=\text{prf}, \text{salt}=\varepsilon, \text{info}=\text{"webauthn-prf-zktv vault key wrap v1"})$
- 4: $b \leftarrow \text{AES-256-GCM-Decrypt}(K_w, \text{nonce}, \text{ciphertext})$; on any failure throw generic `DecryptError` $\triangleright$ never distinguishes wrong key from corrupt data
- 5: reject unless $|b| = 32$; import b as non-extractable AES-256-GCM `CryptoKey` K_v
- 6: zeroize b and prf $\triangleright$ both in `finally` blocks
- 7: **return** (K_v, c') $\triangleright$ caller persists c' for the next unlock

Failure paths are typed: wrong key or tampered ciphertext $\rightarrow$ `DecryptError` (intentionally generic, no cause propagation); malformed or scheme-mismatched records $\rightarrow$ `RecordFormatError`; challenge/origin/counter anomalies $\rightarrow$ `ReplayError`.

4.3 Zero-Knowledge Argument

Formally analyze why stored data R and encrypted vault contents are insufficient to recover secrets without the authenticator. Discuss assumptions on PRF, HKDF, and AES-256-GCM.

5 IndexedDB Storage Architecture

5.1 Schema Overview

The library's `ZktvDb` (raw IndexedDB, no Dexie dependency; database name `zktv`, version 1) creates three object stores:

- **vaults**: composite primary key `[vaultId, scheme]`. Each row holds `vaultId`, `scheme`, the JSON-serialized `WrappedSecretRecord` (version-tagged `v:1`, binary fields base64url-encoded), and `updatedAt`. The composite key lets one vault carry a `prf-v1` record and a `pw-v1` fallback record side by side; `loadWrappedVault` tries `prf-v1` first.
- **credentials**: primary key `credentialId` (base64url), secondary index on `vaultId`. Rows store only non-secret metadata: `vaultId`, the last verified signature `counter`, the base64url PRF salt, authenticator `transports`, and `createdAt`.
- **meta**: key–value store (`keyPath: key`) reserved for application metadata and future migrations.

Every stored row round-trips through `serializeRecord/parseRecord`, so structural invariants (12-byte nonce, ≥ 16 -byte salt, ciphertext longer than the 16-byte

GCM tag, kdfParams present iff pw-v1) are re-validated on every load and violations surface as `RecordFormatError`.

Discrepancy: An earlier draft additionally listed `hibpPrefixes` (breach-detection prefixes) and `sessions` stores. Those belong to the TrustVault-PWA application schema, not to `webauthn-prf-zktv`, whose storage module ships exactly the three stores above (`vaults`, `credentials`, `meta`). If the paper discusses the TrustVault deployment, attribute those stores to the application layer explicitly.

5.2 Schema Diagram

Include a figure depicting the IndexedDB schema and relationships:

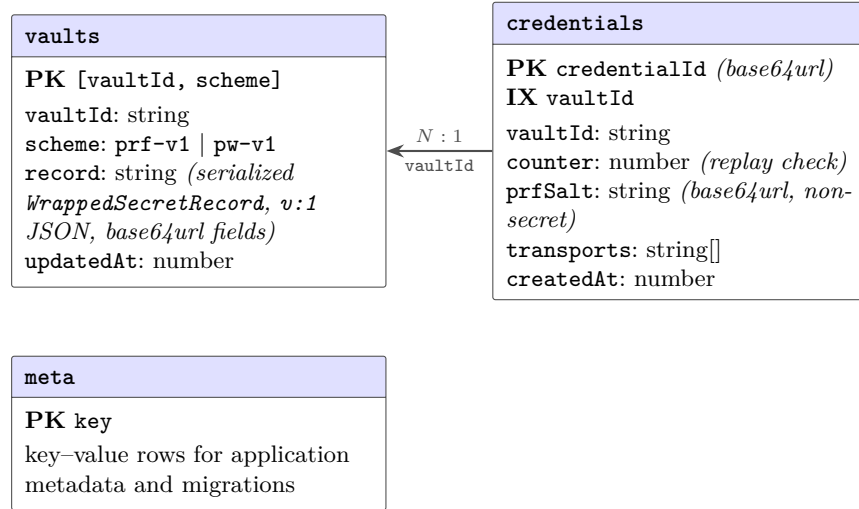


Fig.1. IndexedDB schema created by `openVaultDb` (database `zktv`, version 1; `src/indexeddb/db.ts`). One vault may hold up to two wrapped records (`prf-v1` and the `pw-v1` fallback) via the composite primary key, and any number of PRF credentials via the `vaultId` index. Only wrapped records and non-secret credential metadata are ever persisted.

5.3 Migrations and Security Properties

`openVaultDb` accepts an `onUpgrade(db, oldVersion, tx)` hook that runs inside `onupgradeneeded` after the built-in stores are ensured, letting applications version their own stores without touching the core schema. The migration invariant is that no upgrade may introduce stored recomputable key inputs: only `WrappedSecretRecord` shapes and non-secret credential metadata are ever persisted. Lifecycle operations preserve this: `clearVault` deletes both scheme rows for a vault and every credential row bound to it via the `vaultId` index,

and `securityWipe` clears all three stores (mirroring TrustVault’s security-wipe semantics). Key hygiene throughout: all derived and imported keys are non-extractable `CryptoKeys`, transient buffers are zeroized in `finally` blocks, and the signature counter persisted in `credentials` is updated after each verified assertion (`updateCounter`) to feed the replay check of Algorithm 2.

6 Implementation in webauthn-prf-zktv

The library is TypeScript (strict), ESM-only, with `@noble/hashes` (scrypt) as its sole runtime dependency; all other cryptography uses the Web Crypto API. It ships three sub-path entry points:

- `webauthn-prf-zktv` (core; Node ≥ 20 and browsers): `wrapSecret/unwrapSecret/unwrapSecretBytes`, HKDF and scrypt derivation (`src/core/derive.ts`), record (de)serialization with structural validation (`src/core/serialize.ts`), the TrustVault migration adapter, and the typed error hierarchy (`ZktvError` with codes `PRF_UNSUPPORTED`, `CEREMONY_CANCELLED`, `PRF_RESULT_MISSING`, `REPLAY`, `DECRYPT_FAILED`, `RECORD_FORMAT`, `STORAGE`).
- `webauthn-prf-zktv/webauthn` (browsers only): capability detection and ceremonies. `detectPrfSupport` is tri-state (`supported/unsupported/unknown`) via `PublicKeyCredential.getClientCapabilities()` and never creates throwaway credentials to probe; `isPrfViableOnThisClient` additionally rejects Android WebView (where PRF does not reach the Credential Manager) and checks for a user-verifying platform authenticator. `enrollPrfCredential` is *adaptive*: it requests `prf.eval` at creation time, so clients that evaluate PRF during registration (e.g. Chrome 147+ with Windows Hello) enroll in a single ceremony; otherwise it hard-verifies `prf.enabled` in the extension outputs (aborting with `PrfUnsupportedError` if false) and runs one assertion ceremony to obtain the PRF output. `evaluatePrf` enforces a 32-byte PRF result and performs the client-side replay verification of Algorithm 2. `enrollVault/unlockVault` compose ceremony, HKDF, and (un)wrapping into single calls that zeroize the PRF output in `finally`.
- `webauthn-prf-zktv/indexeddb` (browsers only): the `ZktvDb` storage layer (Fig. 1).

The core wrap entry point, verbatim from `src/core/wrap.ts`:

Listing 1.1. `wrapSecret`: the three wrap paths (PRF, password, caller-supplied wrap key).

```
export async function wrapSecret(options: WrapOptions): Promise<
  WrappedSecretRecord> {
  if ('prfOutput' in options) {
    const wrapKey = await deriveWrapKeyFromPrf(options.prfOutput);
    return encryptRecord(wrapKey, 'prf-v1', options.prfSalt, options.
      secret);
  }
  if ('password' in options) {
```

```

    const salt = options.salt ?? generateSalt();
    const kdfParams = options.kdfParams ?? DEFAULT_SCRYPT_PARAMS;
    const wrapKey = await deriveWrapKeyFromPassword(options.password,
        salt, kdfParams);
    const record = await encryptRecord(wrapKey, 'pw-v1', salt, options.
        secret);
    return { ...record, kdfParams };
}
return encryptRecord(options.wrapKey, options.scheme, options.salt,
    options.secret);
}

```

Listing 1.2. HKDF derivation with the frozen v1 domain-separation label (src/core/derive.ts).

```

export const HKDF_INFO_V1 = new TextEncoder().encode(
    'webauthn-prf-zktv_vault_key_wrap_v1',
);

export async function deriveWrapKeyFromPrf(prfOutput, info = HKDF_INFO_V1) {
    {
        const ikm = await crypto.subtle.importKey('raw', prfOutput, 'HKDF',
            false, ['deriveKey']);
        return crypto.subtle.deriveKey(
            { name: 'HKDF', hash: 'SHA-256', salt: new Uint8Array(0), info },
            ikm,
            { name: 'AES-GCM', length: 256 },
            false, // non-extractable
            ['encrypt', 'decrypt'],
        );
    }
}

```

7 Post-Quantum Considerations

Discuss the quantum-adequate nature of AES-256-GCM and HKDF-SHA256, based on NIST and national guidance. Explain that current WebAuthn passkeys rely on elliptic-curve signatures, which are not post-quantum secure, and outline how PQ-capable passkeys and PRF could integrate with this architecture in the future.

8 Evaluation and Discussion

Outline possible evaluation strategies:

- Security analysis under formal threat models.
- Performance overhead of PRF evaluation and AES operations on typical devices.
- Usability trade-offs between PRF-based biometric unlock and master-password fallback.

9 Conclusion

Summarize the architecture and its contributions. Highlight how the library design enables other developers to adopt PRF-based zero-knowledge vaults with sound IndexedDB storage patterns.

References